

API 연동 가이드 (신규 Nuxt 3 프로젝트용)

BHM 백엔드 API를 새 프로젝트에서 연동할 때 참고하는 문서
API 정의서: `API_SPEC.md` 함께 참고

구현 순서

1. `nuxt.config.ts` → 프록시 설정 (CORS 해결)
2. `.env` → 서버 주소 환경변수
3. `types/response.ts` → API 응답 공통 타입
4. `types/auth.ts` → 로그인 관련 타입
5. `utils/cookie.ts` → 쿠키 읽기/쓰기
6. `composables/useCustomFetch.ts` → `fetch` 래퍼 (토큰 자동 삽입)
7. `composables/useLoginApi.ts` → 로그인 API 호출
8. `stores/authStore.ts` → 전역 로그인 상태 (Pinia)
9. `pages/login.vue` → 로그인 화면

1단계: 프록시 설정

프록시가 필요한 이유 (CORS)

브라우저는 다른 주소의 서버에 직접 요청하는 것을 막습니다 (보안 정책 = CORS).

내 사이트: `http://localhost:3000`
백엔드: `http://192.168.0.99:8080` ← 주소 다름 → 브라우저가 차단

해결책이 프록시 — Nuxt가 중간 우체부 역할:

브라우저 → `localhost:3000/api/auth/login` (같은 주소라 안전)
↓
Nuxt가 대신 전달
↓
`192.168.0.99/api/auth/login` (백엔드)

`nuxt.config.ts`

```
export default defineNuxtConfig({
  runtimeConfig: {
    public: {
```

```

    public: {
      apiBaseUrl: process.env.API_BASE_URL_CLIENT,
    },
  },

  devServer: {
    host: '0.0.0.0',
    port: 3000,
    proxy: {
      '/api': {
        target: process.env.API_PROXY_TARGET || 'http://localhost:8080',
        changeOrigin: true, // 요청 헤더의 origin을 target 주소로 교체
      },
    },
  },
},
})

```

2단계: 환경변수

서버 주소를 코드에 직접 쓰면 개발/운영 환경마다 코드를 수정해야 함 → 환경변수로 분리

프로젝트 루트에 `.env` 파일 생성:

```

# .env
API_BASE_URL_CLIENT=/api
API_PROXY_TARGET=http://192.168.0.99

```

⚠ `.env` 는 절대 git에 올리면 안 됨 → `.gitignore` 에 `.env` 추가 필수

3단계: 타입 정의

왜 필요한가

TypeScript를 쓰는 핵심 이유. API가 어떤 모양의 데이터를 줄지 "계약서"를 써두면:

- 자동완성 제공
- 오타 즉시 에러 표시 (실행 전에 잡힘)

types/response.ts

```

// 모든 API 응답의 공통 구조
export interface BaseResponse {
  success: boolean
  message: string
}

```

```

    httpStatus: number
}

// 단건 응답: data가 하나의 객체
// 예) { success: true, data: { name: "홍길동" } }
export interface SingleResponse<T> extends BaseResponse {
    data: T
}

// 목록 응답: data가 배열
export interface ListResponse<T> extends BaseResponse {
    data: T[]
}

// 페이지네이션 응답 (목록 + 페이지 정보)
export interface PaginationResponse<T> extends BaseResponse {
    data: T[]
    pageInfo: {
        totalPages: number
        totalElements: number
        pageNumber: number
        pageSize: number
        first: boolean
        last: boolean
        empty: boolean
    }
}

```

types/auth.ts

```

// 로그인 요청 body
export interface LoginRequest {
    contactNo: string // 연락처 (로그인 ID)
    password: string
}

// 로그인 성공 응답의 data 부분
export interface LoginResponse {
    accessToken: string
    refreshToken: string
    managerId: string
    managerName: string
    userType: 'ADMIN' | 'COMPANY' | 'DEALER'
    orgNm: string
    orgNo: string
    roleCd: string
    expiresIn: number // 토큰 유효 시간(초)
}

```

4단계: 쿠키 유틸리티

토큰을 쿠키에 저장하는 이유

저장 방법	새로고침 유지	보안
변수(메모리)	❌ 사라짐	-
localStorage	✅	XSS 공격에 취약
쿠키	✅	httpOnly 옵션으로 보안 강화 가능

토큰은 쿠키 저장이 업계 표준.

utils/cookie.ts

```
// 쿠키 저장
// expireSec: 유효 시간(초). 예) 1800 = 30분
export const setCookie = (name: string, value: string, expireSec: number) => {
  document.cookie =
    encodeURIComponent(name) +
    '=' +
    encodeURIComponent(value) +
    ';path=/;Max-Age=' + expireSec + ';'
}

// 쿠키 읽기 (Nuxt의 useCookie 활용 - SSR/CSR 모두 동작)
export const getCookie = (key: string): string | null | undefined => {
  return useCookie(key).value
}

// 쿠키 삭제 (Max-Age=0 으로 즉시 만료)
export const removeCookie = (name: string) => {
  document.cookie = name + '=; path=/; Max-Age=0'
}
```

5단계: fetch 래퍼

래퍼가 필요한 이유

모든 API 호출마다 아래를 반복하면 중복 코드 폭발:

```
// ❌ 이걸 API 수십 개마다 반복?
const token = getCookie('accessToken')
const response = await fetch('/api/users', {
  headers: { 'Authorization': `Bearer ${token}` }
```

```

    headers: { Authorization : Bearer ${token} }
  })
  if (response.status === 401) { /* 토큰 갱신... */ }

```

래퍼로 공통 처리를 한 곳에 모으면:

```

// ✅ 호출할 때는 이것만
const data = await useClientFetch.get('/api/users')

```

composables/useCustomFetch.ts

```

// 모든 요청에 공통 헤더를 삽입하는 내부 함수
const setCommonHeaders = (options: any) => {
  const token = getCookie('accessToken')
  if (token) {
    if (!options.headers) options.headers = new Headers()
    if (options.headers instanceof Headers) {
      options.headers.set('Authorization', `Bearer ${token}`)
    }
  }
}

export const useClientFetch = {
  get: async <T>(url: string, params?: object): Promise<T> => {
    return $fetch<T>(url, {
      method: 'GET',
      params,
      baseURL: useRuntimeConfig().public.apiBaseURL,
      onRequest({ options }) {
        setCommonHeaders(options)
      },
    })
  },

  post: async <T>(url: string, body?: object | FormData): Promise<T> => {
    return $fetch<T>(url, {
      method: 'POST',
      body: body instanceof FormData ? body : JSON.stringify(body),
      baseURL: useRuntimeConfig().public.apiBaseURL,
      onRequest({ options }) {
        setCommonHeaders(options)
      },
    })
  },

  put: async <T>(url: string, body?: object): Promise<T> => {
    return $fetch<T>(url, {
      method: 'PUT',
      body: JSON.stringify(body),
      baseURL: useRuntimeConfig().public.apiBaseURL,

```

```

    onRequest({ options }) {
      setCommonHeaders(options)
    },
  })
},

delete: async <T>(url: string, body?: object): Promise<T> => {
  return $fetch<T>(url, {
    method: 'DELETE',
    body: body ? JSON.stringify(body) : undefined,
    baseUrl: useRuntimeConfig().public.apiBaseUrl,
    onRequest({ options }) {
      setCommonHeaders(options)
    },
  })
},

patch: async <T>(url: string, body?: object): Promise<T> => {
  return $fetch<T>(url, {
    method: 'PATCH',
    body: JSON.stringify(body),
    baseUrl: useRuntimeConfig().public.apiBaseUrl,
    onRequest({ options }) {
      setCommonHeaders(options)
    },
  })
},
}

```

나중에 401/403 응답 시 자동 refresh 로직을 여기에 추가

6단계: 로그인 API composable

composables/useLoginApi.ts

```

import type { SingleResponse } from '~/types/response'
import type { LoginRequest, LoginResponse } from '~/types/auth'

export const useLoginApi = {
  /** 로그인: POST /api/auth/login */
  login: (params: LoginRequest) =>
    useClientFetch.post<SingleResponse<LoginResponse>>('/auth/login', params),

  /** 로그아웃: POST /api/auth/logout */
  logout: () =>
    useClientFetch.post<SingleResponse<string>>('/auth/logout', {}),
}

```

7단계: Auth Store (전역 로그인 상태)

Pinia Store가 필요한 이유

로그인 후 "지금 누가 로그인했는지" 를 앱 전체에서 알아야 함:

- 헤더: "홍길동님 환영합니다" 표시
- 메뉴: 권한에 맞는 항목만 표시
- API 호출: 토큰 읽기

이 정보를 props로 컴포넌트마다 전달하면 유지보수 지옥 → Store = 전역 변수 창고

stores/authStore.ts

```
import { defineStore } from 'pinia'

export const useAuthStore = defineStore('auth', {
  state: () => ({
    isLoggedIn: false,
    managerId: '',
    managerName: '',
    userType: '' as '' | 'ADMIN' | 'COMPANY' | 'DEALER',
    orgNm: '',
    orgNo: '' as string | null,
    accessToken: '',
  }),
  actions: {
    setLogin(data: {
      managerId: string
      managerName: string
      userType: 'ADMIN' | 'COMPANY' | 'DEALER'
      orgNm?: string
      orgNo?: string | null
      accessToken: string
    }) {
      this.isLoggedIn = true
      this.managerId = data.managerId
      this.managerName = data.managerName
      this.userType = data.userType
      this.orgNm = data.orgNm ?? ''
      this.orgNo = data.orgNo ?? ''
      this.accessToken = data.accessToken
    },
    logout() {
      this.isLoggedIn = false
      this.managerId = ''
    }
  }
})
```

```

    this.managerName = ''
    this.userType = ''
    this.orgNm = ''
    this.orgNo = ''
    this.accessToken = ''

    if (import.meta.client) {
      // JS로 제거 가능한 일반 쿠키만 삭제
      // httpOnly refreshToken은 백엔드가 /api/auth/logout 호출 시 만료 처리
      document.cookie = 'accessToken=; path=/; Max-Age=0'
    }
  },
},
},

// persist: true → 새로고침해도 store 상태 유지 (pinia-plugin-persistedstate 필요)
persist: true,
})

```

pinia-plugin-persistedstate 설치 (store 새로고침 유지)

```
npm install pinia-plugin-persistedstate
```

nuxt.config.ts modules에 추가:

```

modules: [
  '@pinia/nuxt',
  'pinia-plugin-persistedstate/nuxt',
]

```

8단계: 로그인 페이지

pages/login.vue

```

<script setup lang="ts">
const form = reactive({
  contactNo: '',
  password: '',
})

const errorMsg = ref('')
const authStore = useAuthStore()

const handleLogin = async () => {
  errorMsg.value = ''

```



```

try {
  const res = await useLoginApi.login(form)

  if (res.success) {
    // accessToken을 쿠키에 저장 (30분 = 1800초)
    setCookie('accessToken', res.data.accessToken, res.data.expiresIn || 1800)
    // refreshToken은 백엔드가 httpOnly 쿠키로 자동 세팅 (JS에서 건드릴 수 없음)

    // 전역 store에 로그인 정보 저장
    authStore.setLogin({
      managerId: res.data.managerId,
      managerName: res.data.managerName,
      userType: res.data.userType,
      orgNm: res.data.orgNm,
      orgNo: res.data.orgNo,
      accessToken: res.data.accessToken,
    })

    // 포털(역할)에 따라 다른 대시보드로 이동
    if (res.data.userType === 'ADMIN') {
      await navigateTo('/admin/dashboard')
    } else if (res.data.userType === 'COMPANY') {
      await navigateTo('/company/dashboard')
    } else if (res.data.userType === 'DEALER') {
      await navigateTo('/dealer/dashboard')
    }
  }
} catch (err: any) {
  const status = err?.response?.status
  if (status === 401) {
    // 로그인 API의 401은 ID/PW 불일치 → 리다이렉트 없이 메시지만 표시
    errorMsg.value = '아이디 또는 비밀번호가 올바르지 않습니다.'
  } else {
    errorMsg.value = '로그인 중 오류가 발생했습니다.'
  }
}
}
</script>

<template>
  <div>
    <input v-model="form.contactNo" placeholder="연락처 (010-xxxx-xxxx)" />
    <input v-model="form.password" type="password" placeholder="비밀번호" />
    <p v-if="errorMsg" style="color: red;">{{ errorMsg }}</p>
    <button @click="handleLogin">로그인</button>
  </div>
</template>

```

토큰 흐름 전체 요약

[로그인]

POST /api/auth/login

↓ 응답

accessToken → 일반 쿠키에 저장 (JS에서 읽기 가능)

refreshToken → httpOnly 쿠키 (JS 접근 불가, 브라우저 자동 전송)

[이후 모든 API 요청]

Header: Authorization: Bearer {accessToken}

[401 또는 403 수신 시] ← accessToken 만료

POST /api/auth/refresh

→ refreshToken은 httpOnly 쿠키라 헤더 불필요, 브라우저가 자동 전송

→ 성공: 새 accessToken 발급 → 원래 요청 1회 재시도

→ 실패: 로그아웃 + 로그인 페이지 이동

다음 단계 (login 완성 후)

단계	내용
401 자동 refresh	<code>useCustomFetch.ts</code> 에 refresh 재시도 로직 추가
페이지 보호	<code>middleware/auth.ts</code> — 로그인 안 한 사용자 차단
역할 보호	<code>middleware/role.ts</code> — 권한 없는 페이지 접근 차단
API 확장	<code>useAdminApi.ts</code> , <code>useCompanyApi.ts</code> 등 추가